# Cahier des charges Backend – PWA Tontine

Nov 2, 2026 · @Akawa

Ce document est destiné à Claude dans VS Code pour construire le backend Supabase de l'application de tontine, compatible avec les types et services définis dans le cahier des charges frontend.

## 1. Objet et consignes pour l'agent

Construis le backend sur Supabase : base PostgreSQL, authentification par SMS, règles d'accès, fonctions serveur, paiements PayDunya, tâches planifiées et notifications push.

**Contexte métier.** Une tontine est un groupe dont les membres cotisent un montant fixe à chaque période ; à chaque tour, un membre reçoit la cagnotte (cotisation × nombre de membres), et chacun la reçoit une seule fois par cycle. L'application a trois rôles globaux ( `super_admin` , `admin` , `user` ) et trois rôles par groupe ( `owner` , `co_admin` , `member` ).

**Consignes de travail.**

- Tout le schéma passe par des migrations SQL versionnées dans `supabase/migrations/` ; ne modifie jamais la base à la main.
- La sécurité repose sur les règles RLS de PostgreSQL, pas sur le frontend : chaque table a RLS activée.
- Les opérations sensibles (démarrer un cycle, tirage, reversement, paiement) passent par des fonctions serveur, jamais par des écritures directes depuis le client.
- Les noms SQL sont en anglais et en `snake_case` ; les messages d'erreur renvoyés à l'utilisateur sont en français.
- Aucun secret dans le code : clés PayDunya, clé VAPID privée et clé `service_role` uniquement dans les secrets Supabase.
- Les contrats d'API doivent correspondre aux types et services du cahier des charges frontend (section 8 de ce document-là). En cas d'écart, signale-le avant de coder.
- Vérifie dans la documentation officielle de PayDunya chaque nom d'endpoint et de champ avant de l'utiliser ; ceux de ce document sont indicatifs.

## 2. Stack et environnement local

Supabase fournit la base, l'authentification, le stockage et les fonctions ; le développement se fait en local avec la CLI Supabase et Docker.

| Brique | Choix | Usage |
|---|---|---|
| Base de données | PostgreSQL (Supabase) | Données, règles RLS, fonctions SQL (RPC) |
| Authentification | Supabase Auth, connexion par téléphone et OTP SMS | Comptes utilisateurs |
| Fournisseur SMS | Twilio (ou autre fournisseur pris en charge par Supabase) | Envoi des codes OTP |
| Stockage | Supabase Storage | Avatars |
| Fonctions serveur | Supabase Edge Functions (Deno, TypeScript) | Paiements, webhook, push, tâches |
| Tâches planifiées | Extensions `pg_cron` et `pg_net` | Ouverture des tours, rappels, retards |
| Paiement | PayDunya (encaissement + déboursement) | Cotisations et reversements |
| Notifications | Web Push (clés VAPID), bibliothèque `web-push` compatible Deno | Push vers la PWA |
| Tests | pgTAP pour SQL, Deno test pour les fonctions | Qualité |

```
npm install -g supabase
supabase init
supabase start            # base locale + Studio sur http://localhost:54323
supabase migration new init_schema
supabase db reset         # rejoue migrations + seed
supabase functions serve  # Edge Functions en local
supabase gen types typescript --local > ../tontine-pwa/src/types/database.ts
```

**Secrets** ( `supabase/functions/.env` en local, `supabase secrets set` en production) : `PAYDUNYA_MASTER_KEY`, `PAYDUNYA_PRIVATE_KEY`, `PAYDUNYA_TOKEN`, `PAYDUNYA_MODE` (test ou live), `VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_SUBJECT`, `APP_URL`, `CRON_SECRET`.

**Organisation du dépôt.**

```
supabase/
  migrations/      # SQL numérotés
  seed.sql         # données de démonstration (mêmes que les mocks du front)
  functions/
    _shared/       # client admin, paydunya.ts, push.ts, errors.ts, cors.ts
    payments-initiate/
    payments-ipn/
    payouts-create/
    payouts-ipn/
    push-send/
    cron-daily/
  tests/           # tests pgTAP
```

## 3. Schéma de base de données

Neuf tables dans le schéma `public`, liées à `auth.users` par `profiles.id` ; tous les montants sont des entiers en FCFA.

```
-- Types
create type global_role as enum ('super_admin','admin','user');
create type group_role as enum ('owner','co_admin','member');
create type frequency as enum ('weekly','biweekly','monthly');
create type group_status as enum ('draft','active','finished');
create type attribution_mode as enum ('random','manual');
create type membership_status as enum ('pending','active','removed');
create type round_status as enum
('upcoming','open','ready_for_payout','paid_out');
create type payment_status as enum
('pending','paid','late','failed','refunded');
create type payment_method as enum
('wave','orange_money','free_money','cash');
create type payout_status as enum
('pending','processing','completed','failed');

create table profiles (
  id uuid primary key references auth.users on delete cascade,
  first_name text not null, last_name text not null,
  phone text unique not null,                -- format +2217XXXXXXXX
  avatar_url text,
  global_role global_role not null default 'user',
  is_suspended boolean not null default false,
  payout_phone text, payout_method payment_method check (payout_method in
('wave','orange_money','free_money')),
  created_at timestamptz not null default now()
);
```

```
create table groups (
  id uuid primary key default gen_random_uuid(),
  name text not null check (char_length(name) between 3 and 50),
  description text,
  contribution_amount int not null check (contribution_amount >= 500),
  frequency frequency not null,
  max_members int not null check (max_members between 2 and 50),
  payment_due_day int not null check (payment_due_day between 1 and 28),
  start_date date not null,
  attribution_mode attribution_mode not null,
  late_penalty int check (late_penalty >= 0),
  status group_status not null default 'draft',
  invite_code text unique not null,           -- 8 caractères, sans 0/O/1/I
  created_by uuid not null references profiles,
  created_at timestamptz not null default now()
);

create table memberships (
  id uuid primary key default gen_random_uuid(),
  group_id uuid not null references groups on delete cascade,
  user_id uuid not null references profiles,
  role group_role not null default 'member',
  status membership_status not null default 'pending',
  payout_order int,                            -- 1..max_members, unique par
groupe
  joined_at timestamptz not null default now(),
  unique (group_id, user_id),
  unique (group_id, payout_order)
);

create table rounds (
  id uuid primary key default gen_random_uuid(),
  group_id uuid not null references groups on delete cascade,
  number int not null,
  open_date date not null, due_date date not null,
  beneficiary_id uuid not null references profiles,
  pot_amount int not null,
  status round_status not null default 'upcoming',
  paid_out_at timestamptz,
  unique (group_id, number)
);

create table payments (
  id uuid primary key default gen_random_uuid(),
  round_id uuid not null references rounds on delete cascade,
```

```
  user_id uuid not null references profiles,
  amount int not null, fee int not null default 0, penalty int not null
default 0,
  method payment_method,
  provider_token text unique,                  -- token de facture PayDunya
  provider_ref text,
  status payment_status not null default 'pending',
  paid_at timestamptz, recorded_by uuid references profiles,  -- espèces
  created_at timestamptz not null default now()
);
-- un seul paiement réussi par membre et par tour
create unique index one_paid_per_round on payments (round_id, user_id) where
status = 'paid';

create table payouts (
  id uuid primary key default gen_random_uuid(),
  round_id uuid unique not null references rounds,
  beneficiary_id uuid not null references profiles,
  amount int not null, phone text not null, method payment_method not null,
  provider_ref text, status payout_status not null default 'pending',
  triggered_by uuid not null references profiles,
  created_at timestamptz not null default now(), completed_at timestamptz
);

create table notifications (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles on delete cascade,
  type text not null, title text not null, body text not null,
  link text, read boolean not null default false,
  created_at timestamptz not null default now()
);

create table push_subscriptions (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles on delete cascade,
  endpoint text unique not null, p256dh text not null, auth text not null,
  created_at timestamptz not null default now()
);

create table audit_log (
  id bigint generated always as identity primary key,
  actor_id uuid references profiles, action text not null,
  entity text not null, entity_id uuid, details jsonb,
  created_at timestamptz not null default now()
);
```

**Compléments.** Index sur toutes les clés étrangères et sur `notifications (user_id, read)`. Trigger `on auth.users insert` qui crée la ligne `profiles` avec le téléphone. Trigger sur `groups insert` qui ajoute le créateur dans `memberships` avec le rôle `owner` et le statut `active`. Bucket Storage `avatars` public en lecture, écriture limitée au dossier `avatars/<user_id>/`.

## 4. Règles d'accès (RLS)

Un utilisateur ne lit que ce qui concerne ses groupes ; seuls les admins d'un groupe le modifient ; le super admin lit tout.

**Fonctions d'aide** (`security definer`, `stable`, `search_path` fixé) :

- `is_super_admin()` : vrai si `profiles.global_role = 'super_admin'` pour `auth.uid()`.
- `is_group_member(gid uuid)` : membre actif du groupe.
- `is_group_admin(gid uuid)` : rôle `owner` ou `co_admin` actif dans le groupe.
- `is_group_owner(gid uuid)` : rôle `owner` dans le groupe.

| Table | Lecture | Création | Modification | Suppression |
|---|---|---|---|---|
| profiles | Soi-même ; membres de mes groupes (sans téléphone, via la vue `group_member_profiles`) ; super admin | Trigger uniquement | Soi-même (sauf `global_role`, `is_suspended`) ; super admin | Interdit |
| groups | Membres actifs ; admins du groupe ; super admin | Rôle global `admin` ou `super_admin` | Admins du groupe, champs verrouillés si `status <> 'draft'` | Propriétaire, si `status = 'draft'` |
| memberships | Membres du groupe ; super admin | Soi-même en `pending` via RPC `request_join` | Admins du groupe (rôle `co_admin` : propriétaire seulement) | Admins du groupe, si `draft` |
| rounds | Membres du groupe ; super admin | Fonction serveur uniquement | Fonction serveur uniquement | Interdit |
| payments | Ses propres paiements ; admins du groupe pour tout le groupe ; statut seul pour les membres (vue `round_payment_status`) | Fonction serveur uniquement | Fonction serveur uniquement | Interdit |

| Table | Lecture | Création | Modification | Suppression |
|---|---|---|---|---|
| payouts | Bénéficiaire ; admins du groupe ; super admin | Fonction serveur uniquement | Fonction serveur uniquement | Interdit |
| notifications | Soi-même | Fonction serveur uniquement | Soi-même ( `read` seulement) | Soi-même |
| push_subscriptions | Soi-même | Soi-même | Soi-même | Soi-même |
| audit_log | Super admin ; admins pour leurs groupes | Fonction serveur uniquement | Interdit | Interdit |

Un compte avec `is_suspended = true` est bloqué : toutes les fonctions RPC vérifient ce drapeau et renvoient l'erreur `compte_suspendu`. Les numéros de téléphone des membres ne sont visibles que des admins du groupe.

## 5. API

Les lectures simples passent par le client Supabase (tables et vues protégées par RLS) ; toute action métier passe par une fonction RPC SQL ou une Edge Function.

**Fonctions RPC (SQL, `security definer`, vérification des droits en tête).**

| Fonction | Qui | Effet |
|---|---|---|
| `create_group(payload jsonb)` | admin, super_admin | Crée le groupe en `draft`, génère `invite_code`, ajoute le créateur en `owner` ; renvoie le groupe |
| `get_group_by_invite(code text)` | Connecté | Aperçu public : nom, montant, fréquence, places restantes, admins |
| `request_join(code text)` | Connecté | Crée une adhésion `pending` ; refuse si groupe complet ou démarré |
| `review_membership(id uuid, accept boolean)` | Admin du groupe | Accepte ou refuse ; notifie le demandeur |
| `add_member_by_phone(gid uuid, phone text)` | Admin du groupe | Ajoute directement un utilisateur existant en `active` |

| Fonction | Qui | Effet |
|---|---|---|
| `remove_member(id uuid)` | Admin du groupe | Retire un membre, seulement en `draft` |
| `set_member_role(id uuid, role group_role)` | Propriétaire | Passe un membre en `co_admin` ou le rétrograde |
| `draw_payout_order(gid uuid)` | Admin du groupe | Tirage aléatoire côté serveur, enregistré dans `audit_log`, notifie tous les membres |
| `set_manual_order(gid uuid, order uuid[])` | Admin du groupe | Fixe l'ordre manuellement, en `draft` seulement |
| `start_cycle(gid uuid)` | Admin du groupe | Vérifie groupe complet et ordre fixé, crée tous les tours, passe en `active` |
| `record_cash_payment(round_id uuid, user_id uuid)` | Admin du groupe | Crée un paiement `paid` en espèces |
| `remind_late(round_id uuid)` | Admin du groupe | Notifie les membres non payés (1 fois par 24 h maximum) |
| `admin_stats()` | admin, super_admin | Indicateurs du tableau de bord, limités à ses groupes pour un admin |
| `platform_stats()` | super_admin | Utilisateurs, groupes, total collecté |
| `set_global_role(user_id uuid, role global_role)` | super_admin | Nomme ou retire un administrateur |
| `set_suspended(user_id uuid, value boolean)` | super_admin | Suspend ou réactive un compte |
| `mark_all_notifications_read()` | Connecté | Marque toutes ses notifications comme lues |

### Edge Functions (HTTP).

| Fonction | Appelée par | Rôle |
|---|---|---|
| `payments-initiate` | Front (JWT utilisateur) | Crée la facture PayDunya et renvoie `{ paymentId, checkoutUrl }` |

| Fonction | Appelée par | Rôle |
|---|---|---|
| `payments-ipn` | PayDunya (sans JWT) | Reçoit la confirmation, vérifie, met à jour le paiement |
| `payouts-create` | Front (JWT admin du groupe) | Lance le reversement de la cagnotte |
| `payouts-ipn` | PayDunya (sans JWT) | Confirme le reversement |
| `push-send` | Interne (clé `service_role`) | Envoie un push à un ou plusieurs utilisateurs |
| `cron-daily` | `pg_cron` (en-tête `CRON_SECRET`) | Ouvre les tours, rappels, retards |

**Erreurs.** Chaque fonction renvoie un code stable (`groupe_complet`, `cycle_deja_demarre`, `non_autorise`, `tour_non_pret`, `paiement_deja_effectue`, `compte_suspendu`) et un message en français que le front affiche tel quel.

## 6. Logique métier

Un groupe passe de brouillon à actif puis terminé ; chaque tour passe de à venir à ouvert, prêt pour reversement, puis reversé.

**Cycle d'un groupe.**

1. `draft` : configuration modifiable, adhésions acceptées jusqu'à `max_members` membres actifs.
2. `start_cycle` exige : membres actifs = `max_members`, un `payout_order` unique pour chacun, chaque membre a un numéro de reversement.
3. `start_cycle` crée `max_members` tours : tour n° k, bénéficiaire = membre d'ordre k, `pot_amount = contribution_amount × max_members`.
4. Dates : `open_date` du tour 1 = `start_date` ; tours suivants + 7 jours (weekly), + 14 jours (biweekly) ou + 1 mois (monthly). `due_date` = jour `payment_due_day` de la période pour monthly, `open_date` + 5 jours sinon.
5. Le groupe passe à `active` ; montant, fréquence, nombre de membres et ordre sont verrouillés (trigger qui refuse la modification).
6. Quand le dernier tour est `paid_out`, le groupe passe à `finished`.

**Statuts d'un tour.**

| Statut | Condition d'entrée | Déclenché par |
|---|---|---|
| `upcoming` | Création par `start_cycle` | `start_cycle` |
| `open` | `open_date` atteinte ; un paiement `pending` est créé pour chaque membre | `cron-daily` |
| `ready_for_payout` | Tous les paiements du tour sont `paid` | Trigger sur `payments` |
| `paid_out` | Reversement confirmé par PayDunya | `payouts-ipn` |

**Paiements en retard.** Le lendemain de `due_date`, `cron-daily` passe les paiements `pending` en `late` et ajoute `late_penalty` dans `penalty` si le groupe en a une. Un paiement `late` peut toujours être payé ; il passe alors à `paid`.

**Règles invariantes** (à tester) :

- Un membre reçoit la cagnotte une seule fois par cycle.
- Un membre ne paie jamais deux fois le même tour (index unique).
- Le montant payé = `contribution_amount` + `penalty` ; tout autre montant reçu est rejeté et journalisé.
- Un reversement n'est possible que si le tour est `ready_for_payout` et n'a pas déjà de `payouts`.
- Toute modification sensible écrit une ligne dans `audit_log`.

## 7. Paiements et reversements

Le backend crée la facture chez PayDunya, et seule une confirmation vérifiée côté serveur fait passer un paiement à `paid`.

**Encaissement d'une cotisation (`payments-initiate` puis `payments-ipn`).**

1. Le front appelle `payments-initiate` avec `{ roundId, method }`.
2. La fonction vérifie : utilisateur membre actif, tour `open` ou paiement `late`, pas déjà payé. Montant = `contribution_amount` + `penalty`.
3. Elle crée (ou réutilise) la ligne `payments` en `pending`, puis appelle l'API de facture PayDunya avec le montant, une description « Cotisation [groupe] – tour [n] », `custom_data` = `{ payment_id }`, `return_url` = `APP_URL/paiement/succes?payment=<id>`, `cancel_url` = `APP_URL/paiement/echec?payment=<id>`, `callback_url` = URL de `payments-ipn`.
4. Elle enregistre le token de facture dans `provider_token` et renvoie l'URL de paiement.

5. `payments-ipn` reçoit la notification, vérifie son authenticité (empreinte fournie par PayDunya comparée à celle de la clé maître, selon leur documentation), puis **reconfirme le statut** en interrogeant l'API PayDunya avec le token avant toute mise à jour.
6. Si la facture est payée et le montant correspond : `status = 'paid'`, `paid_at`, `method`, `provider_ref` ; notification au membre ; trigger de passage du tour à `ready_for_payout` si tous ont payé, avec notification aux admins.
7. Idempotence : si le paiement est déjà `paid`, répondre 200 sans rien changer. Toujours répondre 200 à PayDunya une fois la notification traitée, et journaliser chaque appel.

**Reversement de la cagnotte (`payouts-create` puis `payouts-ipn`).**

1. Un admin du groupe appelle `payouts-create` avec `{ roundId }`.
2. Vérifications : tour `ready_for_payout`, aucun `payouts` existant, bénéficiaire avec numéro de reversement.
3. Création de la ligne `payouts` en `processing`, puis appel de l'API de déboursement PayDunya (création puis soumission de la facture de déboursement) vers le portefeuille du bénéficiaire.
4. `payouts-ipn` confirme : `payouts.status = 'completed'`, `rounds.status = 'paid_out'`, notification à tous les membres ; en cas d'échec, `failed` et notification aux admins.

**Mode test.** `PAYDUNYA_MODE=test` utilise l'environnement de test de PayDunya. En local, ajouter une route `payments-ipn?simulate=paid` acceptée uniquement si `PAYDUNYA_MODE=test`, pour simuler une confirmation sans passer par PayDunya.

**Frais.** Le montant facturé au membre et les frais prélevés par PayDunya sont stockés séparément (`amount`, `fee`) pour permettre la réconciliation. La règle « qui paie les frais » reste à trancher ; prévoir un réglage par groupe `fees_paid_by` (`member` ou `group`) sans l'exposer dans l'interface pour l'instant.

Documentation : API de déboursement PayDunya.

## 8. Tâches planifiées et notifications

Une tâche quotidienne fait avancer les tours et envoie les rappels ; chaque notification est à la fois enregistrée en base et envoyée en push.

**Tâche `cron-daily`** (via `pg_cron`, tous les jours à 08:00 heure de Dakar, soit 08:00 UTC) :

1. Ouvre les tours dont `open_date` = aujourd'hui, crée les paiements `pending` et notifie « Le tour [n] de [groupe] est ouvert. Bénéficiaire : [nom] ».
2. Rappel J-3 et J-1 avant `due_date` aux membres non payés.

3. Passe en `late` les paiements `pending` dont `due_date` est dépassée et notifie le membre et les admins.
4. Signale aux admins les tours `ready_for_payout` depuis plus de 48 h.

La tâche est idempotente : relancée deux fois le même jour, elle ne crée ni doublon de paiement ni doublon de notification (clé de dédoublonnage `type + user_id + entité + date`).

**Fonction SQL `notify(user_ids uuid[], type text, title text, body text, link text)`.** Insère les lignes dans `notifications`, puis appelle `push-send` via `pg_net`. Toutes les notifications du système passent par elle.

| Type | Destinataires | Lien |
|---|---|---|
| `round_opened` | Membres du groupe | `/groupes/:id` |
| `payment_reminder` | Membres non payés | `/groupes/:id/payer` |
| `payment_late` | Membre + admins | `/groupes/:id` |
| `payment_confirmed` | Membre | `/groupes/:id` |
| `round_ready` | Admins du groupe | `/admin/groupes/:id` |
| `payout_done` | Membres du groupe | `/groupes/:id` |
| `payout_failed` | Admins du groupe | `/admin/groupes/:id` |
| `join_request` | Admins du groupe | `/admin/groupes/:id` |
| `join_reviewed` | Demandeur | `/groupes/:id` |
| `order_drawn` | Membres du groupe | `/groupes/:id` |
| `cycle_finished` | Membres du groupe | `/groupes/:id` |

**`push-send`.** Lit les `push_subscriptions` des destinataires, envoie le message avec les clés VAPID (titre, corps, lien, icône), et supprime les abonnements qui répondent 404 ou 410 (appareil désinscrit). Respecte les préférences de notification de l'utilisateur (colonne `notification_prefs jsonb` à ajouter dans `profiles`).

## 9. Sécurité et journal d'audit

L'argent circule : chaque action financière est vérifiée côté serveur, traçable et impossible à rejouer.

- **Authentification** : OTP SMS uniquement, limite de 5 demandes de code par numéro et par heure (réglage Supabase Auth), sessions avec jeton d'accès court et rafraîchissement.
- **RLS partout** : test automatique qui échoue si une table de `public` n'a pas RLS activée.
- **Fonctions `security definer`** : `search_path` fixé à `public`, vérification explicite de `auth.uid()` et des droits en première ligne, aucune construction de SQL par concaténation.
- **Webhooks** : authenticité vérifiée, puis statut reconfirmé auprès de PayDunya ; une notification falsifiée ne doit jamais marquer un paiement comme payé.
- **Clé `service_role`** : utilisée uniquement dans les Edge Functions, jamais renvoyée au client.
- **CORS** : Edge Functions appelées par le front limitées à `APP_URL` et `http://localhost:5173`.
- **Limitation** : `payments-initiate` limité à 10 appels par utilisateur et par minute ; `remind_late` à 1 fois par 24 h par tour.
- **Données personnelles** : téléphones visibles des seuls admins du groupe ; suppression de compte possible (anonymisation du profil, conservation des paiements pour la comptabilité), dans l'esprit de la loi sénégalaise sur les données personnelles.
- **Sauvegardes** : sauvegardes quotidiennes Supabase activées en production.

**Journal d'audit.** Une ligne dans `audit_log` pour : création et démarrage d'un groupe, tirage au sort (avec l'ordre obtenu), modification d'ordre, acceptation, refus et retrait de membre, changement de rôle, paiement en espèces, paiement confirmé, reversement déclenché et confirmé, nomination ou suspension d'un admin, tout montant reçu incohérent. Le journal est en ajout seul (aucune modification ni suppression, même par le super admin).

## 10. Environnements, tests et déploiement

Trois environnements : local pour développer, test pour le pilote avec un vrai groupe, production pour le public.

| Environnement | Base | PayDunya | SMS | Usage |
|---|---|---|---|---|
| Local | `supabase start` (Docker) | Mode test + simulation | OTP de test Supabase (numéros fixes) | Développement |

| Environnement | Base | PayDunya | SMS | Usage |
|---|---|---|---|---|
| Test | Projet Supabase « tontine-test » | Mode test | Fournisseur SMS réel | Pilote avec un groupe |
| Production | Projet Supabase « tontine-prod » | Mode live | Fournisseur SMS réel | Public |

**Données de démonstration.** `seed.sql` reprend exactement les comptes et groupes des mocks du frontend (1 super admin, 2 admins, 12 participants, 3 groupes) pour que le front bascule de `VITE_USE_MOCKS=true` à `false` sans surprise. En local, configurer des numéros de test Supabase Auth avec le code `123456`.

**Tests à écrire.**

- pgTAP : RLS de chaque table pour chaque rôle (un participant ne lit pas un autre groupe, un co-admin ne supprime pas, un admin ne voit pas les groupes des autres, seul le super admin nomme des admins).
- pgTAP : `start_cycle` (dates, bénéficiaires, montants), verrouillage après démarrage, unicité du paiement, passage automatique à `ready_for_payout`.
- Deno test : `payments-ipn` (notification valide, falsifiée, en double, montant incohérent), `cron-daily` (idempotence), `push-send` (abonnement expiré supprimé).

**Déploiement.** GitHub Actions : sur chaque pull request, `supabase db reset` + tests ; sur la branche `main`, `supabase db push` et `supabase functions deploy` vers le projet de test ; mise en production par tag de version. Les secrets sont stockés dans GitHub et dans Supabase, jamais dans le dépôt.

## 11. Ordre de travail et critères d'acceptation

Travaille dans cet ordre et arrête-toi à la fin de chaque étape pour que je puisse tester.

1. **Initialisation** : projet Supabase local, structure du dépôt, `.env.example`, README avec les commandes.
2. **Schéma** : migrations des types, tables, index et triggers de la section 3 ; génération des types TypeScript pour le front.
3. **RLS** : fonctions d'aide, politiques de la section 4, vues `group_member_profiles` et `round_payment_status`, tests pgTAP.
4. **Authentification** : connexion par téléphone, trigger de profil, numéros de test, seed des comptes de démonstration.

5. **Groupes et membres** : RPC de création, invitation, adhésion, rôles, tirage et ordre manuel.
6. **Cycle** : `start_cycle`, verrouillage, transitions de statut, tests.
7. **Notifications** : fonction `notify`, `push-send`, abonnements push.
8. **Tâche quotidienne** : `cron-daily` et sa planification `pg_cron`.
9. **Paiements** : `payments-initiate`, `payments-ipn`, mode simulation, tests.
10. **Reversements** : `payouts-create`, `payouts-ipn`.
11. **Super admin et statistiques** : RPC d'administration et de statistiques.
12. **Déploiement** : projet de test, secrets, GitHub Actions.

**Critères d'acceptation.**

- [ ] `supabase db reset` rejoue toutes les migrations et le seed sans erreur ; tous les tests passent.
- [ ] Aucune table de `public` sans RLS ; un participant ne peut lire aucune donnée d'un groupe dont il n'est pas membre.
- [ ] Un admin crée un groupe de 4 500 FCFA à 10 membres ; après `start_cycle`, il existe 10 tours avec une cagnotte de 45 000 FCFA et 10 bénéficiaires distincts.
- [ ] Une fois le cycle démarré, toute tentative de modifier le montant, la fréquence ou le nombre de membres est refusée.
- [ ] En mode test, un paiement simulé passe à `paid`, le membre reçoit une notification, et le tour passe à `ready_for_payout` quand tous ont payé.
- [ ] Une notification de paiement envoyée deux fois, ou falsifiée, ne change rien.
- [ ] Un reversement ne peut être lancé qu'une fois par tour ; après confirmation, le tour est `paid_out` et tous les membres sont notifiés.
- [ ] Relancer `cron-daily` deux fois le même jour ne crée aucun doublon.
- [ ] Seul le super admin peut nommer, suspendre ou retirer un administrateur ; un compte suspendu ne peut plus rien faire.
- [ ] Le frontend fonctionne avec `VITE_USE_MOCKS=false` sur les mêmes parcours qu'en mode mock.

**Hors périmètre.** SMS de secours pour les notifications, reversement automatique sans action de l'admin, export PDF et Excel, tableau de bord comptable, traduction en wolof.